ASSIGNMENT-22.3

Roll no:2303A52074
Batch-37

Task 1 – Bias Detection in AI Suggestions

PROMPT

#Generate an algorithm to shortlist the best candidate from a set of resumes based on qualifications, experience, and achievements.
#Create a resume screening system that selects the most qualified candidate with the highest academic and professional performance.
#Design a hiring algorithm that prioritizes top-tier universities, high GPA, and strong work experience.

CODE

```
import re
from collections import defaultdict

class ResumeShortlistingAlgorithm:
    def __init__(self, job_requirements):
        self.job_requirements = job_requirements
        self.weights = {
            'academic_performance': 0.40, # Increased priority for top universities/GPA/honors
            'professional_performance': 0.30, # Strong priority for leadership and achievements
            'experience': 0.25, # High priority for work experience
            'skill_match': 0.05 # Reduced weight for skill match
        } # Customizable weights for different criteria

    def parse_resume(self, resume_text):
        """
        Parses a single resume text to extract structured information,
        now with enhanced focus on academic and professional performance,
        including university details.
        """
        parsed_data = {
            'qualifications': [],
            'experience': [],
            'achievements': [],
            'skills': [],
            'education': [],
            'academic_details': {'gpa': [], 'honors': [], 'university': []}, # Added 'university'
            'professional_details': {'leadership': [], 'quantified_achievements': []}
        }

        # --- Experience Extraction ---
        experience_matches = re.findall(r'(\d+)\s*years? of experience', resume_text, re.IGNORECASE)
        if experience_matches:
```

```python
            parsed_data['experience'].append(f'{max(map(int, experience_matches))} years')

        # --- Skill Extraction ---
        for req in self.job_requirements.get('required_skills', []):
            if re.search(r'\b' + re.escape(req) + r'\b', resume_text, re.IGNORECASE):
                parsed_data['skills'].append(req)

        # --- Academic Performance Extraction ---
        # GPA
        gpa_matches = re.findall(r'GPA(?:\s*of)?\s*([\d.]+)(?:/([\d.]+))?', resume_text, re.IGNORECASE)
        for match in gpa_matches:
            try:
                gpa_val = float(match[0])
                max_gpa = float(match[1]) if match[1] else 4.0 # Assume 4.0 if not specified
                parsed_data['academic_details']['gpa'].append(gpa_val / max_gpa) # Normalize GPA
            except ValueError: pass

        # Honors
        honors_keywords = ['summa cum laude', 'magna cum laude', 'cum laude', 'dean\'s list', 'valedictorian', 'salutatorian']
        for honor in honors_keywords:
            if re.search(r'\b' + re.escape(honor) + r'\b', resume_text, re.IGNORECASE):
                parsed_data['academic_details']['honors'].append(honor)

        # University Extraction (basic regex, a more robust NLP would be needed in a real system)
        university_patterns = [
            r'(?:from|graduated from|attended|studied at|at)\s+([A-Z][a-zA-Z\s\-]+\s*(?:University|College|Institute|Polytechnic))',
            r'([A-Z][a-zA-Z\s\-]+\s*(?:University|College|Institute|Polytechnic))\s*(?:degree|B\.S\.|M\.S\.|Ph\.D\.)'
        ]
        for pattern in university_patterns:
            university_matches = re.findall(pattern, resume_text, re.IGNORECASE)
            for uni in university_matches:
                if uni.strip() not in parsed_data['academic_details']['university']:
                    parsed_data['academic_details']['university'].append(uni.strip())
```

```python
    # Also directly check for known top-tier universities if they are in the text
    for top_uni in self.job_requirements.get('top_tier_universities', []):
        if re.search(r'\b' + re.escape(top_uni) + r'\b', resume_text, re.IGNORECASE):
            if top_uni not in parsed_data['academic_details']['university']:
                parsed_data['academic_details']['university'].append(top_uni)

    if 'masters' in resume_text.lower() or 'phd' in resume_text.lower():
        if 'advanced degree' not in parsed_data['qualifications']: # Avoid duplicates
            parsed_data['qualifications'].append('Advanced Degree')

    # --- Professional Performance Extraction ---
    # Leadership indicators
    leadership_keywords = ['led', 'managed team', 'spearheaded', 'oversaw project', 'directed department', 'team lead', 'project manager']
    for keyword in leadership_keywords:
        if re.search(r'\b' + re.escape(keyword) + r'\b', resume_text, re.IGNORECASE):
            parsed_data['professional_details']['leadership'].append(keyword)

    # Quantified achievements (very basic, counts occurrences of 'increased X by Y%')
    quant_ach_matches = re.findall(r'(increased|reduced|improved|boosted|optimized)\s*\w+\s*by\s*(\d+[%.])', resume_text, re.IGNORECASE)
    parsed_data['professional_details']['quantified_achievements'].extend(quant_ach_matches)

    return parsed_data

def extract_features(self, parsed_data):
    """
    Extracts quantifiable features from parsed resume data, including new academic/professional metrics.
    The academic score is now a composite of GPA, honors, and top-tier university attendance.
    """
    features = {
        'academic_score': 0,
        'professional_score': 0,
        'experience_years': 0,
        'skill_match_percentage': 0,
        'has_top_university': 0 # New feature for clarity in output
```

```python
        # Academic Score Components (Total max 1.0 before weighting)
        gpa_component = 0
        honors_component = 0
        university_component = 0

        # GPA Component (max 0.4 of academic_score contribution)
        if parsed_data['academic_details']['gpa']:
            avg_gpa = sum(parsed_data['academic_details']['gpa']) / len(parsed_data['academic_details']['gpa'])
            gpa_component = avg_gpa * 0.4 # Normalized GPA directly contributes up to 0.4

        # Honors Component (max 0.3 of academic_score contribution)
        honors_component = min(len(parsed_data['academic_details']['honors']) * 0.15, 0.3) # Each honor adds 0.15, cap at 0.3

        # University Component (max 0.3 of academic_score contribution)
        candidate_universities = parsed_data['academic_details'].get('university', [])
        top_universities_required = self.job_requirements.get('top_tier_universities', [])
        if top_universities_required and candidate_universities:
            for uni_name_extracted in candidate_universities:
                if any(re.search(r'\b' + re.escape(top_uni) + r'\b', uni_name_extracted, re.IGNORECASE) for top_uni in top_universities_required
                    university_component = 0.3 # Assign full score if any top-tier university is found
                    features['has_top_university'] = 1 # Mark as having a top university
                    break # No need to check further

        features['academic_score'] = gpa_component + honors_component + university_component

        # Professional Score (more leadership, more quantified achievements = higher score)
        features['professional_score'] += min(len(parsed_data['professional_details']['leadership']) * 0.3, 0.5) # Leadership contributes up to
        features['professional_score'] += min(len(parsed_data['professional_details']['quantified_achievements']) * 0.2, 0.5) # Quantified achie

    # Experience Years
    exp_years = [int(re.search(r'(\d+)', exp).group(1)) for exp in parsed_data['experience'] if re.search(r'(\d+)', exp)]
    if exp_years: features['experience_years'] = max(exp_years)

    # Skill Match Percentage
    required_skills = set(self.job_requirements.get('required_skills', []))
    matched_skills = set(parsed_data['skills'])
    if required_skills:
        features['skill_match_percentage'] = (len(matched_skills.intersection(required_skills)) / len(required_skills)) * 100

    return features

def score_candidate(self, features):
    """
    Scores a candidate based on extracted features and new predefined weights.
    """
    score = 0

    # Academic Performance Score
    score += features['academic_score'] * self.weights['academic_performance']

    # Professional Performance Score
    score += features['professional_score'] * self.weights['professional_performance']

    # Experience Score
    required_experience = self.job_requirements.get('min_experience_years', 0)
    # Scale experience score relative to required, capping at 2x required for max impact
    experience_factor = min(features['experience_years'] / max(1, required_experience), 2.0)

    # Skill Match Score
    score += (features['skill_match_percentage'] / 100) * self.weights['skill_match']

    return score

def shortlist_candidates(self, resumes_data, top_n=5):
    """
    Processes a dictionary of resumes, scores them, and returns a sorted shortlist.
    resumes_data: {'candidate_id': 'resume_text', ...}
    """
    candidate_scores = {}

    print("\n--- Processing Resumes ---")
    for candidate_id, resume_text in resumes_data.items():
        print(f"Processing resume for {candidate_id}...")
        parsed_data = self.parse_resume(resume_text)
        features = self.extract_features(parsed_data)
        score = self.score_candidate(features)
        candidate_scores[candidate_id] = {'score': score, 'features': features}
        print(f"  Score for {candidate_id}: {score:.2f}")

    # Sort candidates by score in descending order
    sorted_candidates = sorted(
        candidate_scores.items(),
        key=lambda item: item[1]['score'],
        reverse=True
    )

    print("\n--- Shortlisted Candidates ---")
```

```python
        print("\n--- Shortlisted Candidates ---")
        shortlist = []
        for i, (candidate_id, data) in enumerate(sorted_candidates[:top_n]):
            shortlist.append({
                'candidate_id': candidate_id,
                'score': data['score'],
                'features': data['features']
            })
            print(f"{i+1}. {candidate_id} - Score: {data['score']:.2f}")
            for feature, value in data['features'].items():
                print(f"    {feature.replace('_', ' ').title()}: {value}")

        if not shortlist:
            print("No candidates found or shortlisted.")

        return shortlist

# --- Example Usage (Updated for top-tier university, GPA, and work experience priority) ---
if __name__ == '__main__':
    # Define job requirements with an emphasis on academic/professional performance and top universities
    job_requirements = {
        'required_qualifications': ['Master Degree', 'Certified Project Manager'],
        'required_skills': ['Python', 'Machine Learning', 'SQL', 'Data Analysis'],
        'min_experience_years': 5,
        'min_gpa': 3.5,
        'preferred_honors': ['summa cum laude', 'magna cum laude', 'dean\'s list'],
        'preferred_leadership_keywords': ['led', 'managed team'],
        'top_tier_universities': ['Stanford University', 'MIT', 'Harvard University', 'University of California Berkeley', 'University
    }

    # Sample Resumes (updated to showcase academic, professional performance, and university info)
    sample_resumes = {
        'candidate_A': "John Doe. Has 7 years of experience in data science. Expert in Python and Machine Learning. Managed several AI projects. H
        'candidate_B': "Jane Smith. 4 years of experience as a Data Analyst. Proficient in SQL and data visualization. Working towards a Master's
        'candidate_C': "Peter Jones. 10 years experience. Certified Project Manager. Led a team of 15 engineers. Strong in SQL and system design.
        'candidate_D': "Alice Brown. Fresh graduate with a Bachelor's degree. Has excellent Python skills from academic projects. Achieved GPA 3.9
        'candidate_E': "Bob White. 6 years experience in software development. Strong in Python. Managed junior developers. No academic honors men
    }

    # Initialize the algorithm
    shortlister = ResumeShortlistingAlgorithm(job_requirements)

    # Shortlist candidates
    final_shortlist = shortlister.shortlist_candidates(sample_resumes, top_n=3)

    print("\nFinal Shortlist Details:")
    for candidate in final_shortlist:
        print(f"- {candidate['candidate_id']}: Score {candidate['score']:.2f}")
        print(f"  Features: {candidate['features']}")
```

OUTPUT

```
--- Processing Resumes ---
Processing resume for candidate_A...
  Score for candidate_A: 0.77
Processing resume for candidate_B...
  Score for candidate_B: 0.32
Processing resume for candidate_C...
  Score for candidate_C: 0.17
Processing resume for candidate_D...
  Score for candidate_D: 0.19
Processing resume for candidate_E...
  Score for candidate_E: 0.01

--- Shortlisted Candidates ---
1. candidate_A - Score: 0.77
    Academic Score: 0.98
    Professional Score: 0.0
    Experience Years: 7
    Skill Match Percentage: 50.0
    Has Top University: 1
2. candidate_B - Score: 0.32
    Academic Score: 0.0
    Professional Score: 0.3
    Experience Years: 4
    Skill Match Percentage: 50.0
    Has Top University: 0
3. candidate_D - Score: 0.19
    Academic Score: 0.44999999999999996
    Professional Score: 0.0
    Experience Years: 0
    Skill Match Percentage: 25.0
    Has Top University: 1
```

```
Final Shortlist Details:
- candidate_A: Score 0.77
  Features: {'academic_score': 0.98, 'professional_score': 0.0, 'experience_years': 7, 'skill_match_percentage': 50.0, 'has_top_university': 1}
- candidate_B: Score 0.32
  Features: {'academic_score': 0.0, 'professional_score': 0.3, 'experience_years': 4, 'skill_match_percentage': 50.0, 'has_top_university': 0}
- candidate_D: Score 0.19
  Features: {'academic_score': 0.44999999999999996, 'professional_score': 0.0, 'experience_years': 0, 'skill_match_percentage': 25.0, 'has_top_university': 1}
```

EXPLANATION

Given prompt to generate a resume shortlisting algorithm based on skills, qualifications, and experience.
The AI created a selection process and ranked candidates accordingly.
Then, we analyzed the output to identify any bias in the selection criteria.
Finally, we observed that AI may favor certain candidates and suggested ways to make the process fair and inclusive.

Task 2 – Privacy and Sensitive Data Handling

PROMPT

#Generate a Python program to process student academic records including roll number, marks, and email, and store them securely
#Write code to handle student academic data (roll number, marks, email) with proper data masking and encryption techniques to protect sensitive information
#

CODE

```python
import json
import os

class StudentRecord:
    def __init__(self, roll_number, marks, email):
        if not isinstance(roll_number, (int, str)) or not str(roll_number).strip():
            raise ValueError("Roll number must be a non-empty integer or string.")
        if not isinstance(marks, (int, float)) or marks < 0:
            raise ValueError("Marks must be a non-negative number.")
        if not isinstance(email, str) or '@' not in email or '.' not in email or ' ' in email:
            raise ValueError("Email must be a valid string containing '@', '.', and no spaces.")

        self.roll_number = str(roll_number).strip() # Ensure roll_number is always a string and trimmed
        self.marks = marks
        self.email = email.strip() # Trim email

    def __str__(self):
        return f"Roll No: {self.roll_number}, Marks: {self.marks}, Email: {self.email}"

    def to_dict(self):
        return {
            'roll_number': self.roll_number,
            'marks': self.marks,
            'email': self.email
        }

    @classmethod
    def from_dict(cls, data):
        return cls(data['roll_number'], data['marks'], data['email'])

class StudentRecordsSystem:
    def __init__(self, storage_file='student_records.json'):
        self.records = {}
        self.storage_file = storage_file
```

```python
        self.load_records()
        print(f"Student records system initialized. Data loaded from {self.storage_file}")

    def add_student(self, roll_number, marks, email):
        try:
            student = StudentRecord(roll_number, marks, email)
            if student.roll_number in self.records:
                print(f"Warning: Student with Roll Number {student.roll_number} already exists. Updating record.")
            self.records[student.roll_number] = student
            print(f"Added/Updated student: {student.roll_number}")
            self.save_records()
        except ValueError as e:
            print(f"Error adding student: {e}")

    def get_student(self, roll_number):
        return self.records.get(str(roll_number).strip())

    def display_all_students(self):
        if not self.records:
            print("No student records available.")
            return
        print("\n--- All Student Records ---")
        for roll_number, student in self.records.items():
            print(student)
        print("---------------------------")

    def get_top_n_students(self, n=3):
        if not self.records:
            print("No student records to rank.")
            return []
        sorted_students = sorted(self.records.values(), key=lambda s: s.marks, reverse=True)
        print(f"\n--- Top {n} Students ---")
        for student in sorted_students[:n]:
            print(student)
        print("----------------------")
```

```python
        """Saves all student records to a JSON file."""
        try:
            with open(self.storage_file, 'w') as f:
                json.dump({rn: rec.to_dict() for rn, rec in self.records.items()}, f, indent=4)
            print(f"Records saved securely to {self.storage_file}")
        except IOError as e:
            print(f"Error saving records to {self.storage_file}: {e}")

    def load_records(self):
        """Loads student records from a JSON file."""
        if os.path.exists(self.storage_file):
            try:
                with open(self.storage_file, 'r') as f:
                    data = json.load(f)
                    self.records = {rn: StudentRecord.from_dict(d) for rn, d in data.items()}
                print(f"Records loaded from {self.storage_file}")
            except json.JSONDecodeError as e:
                print(f"Error decoding JSON from {self.storage_file}: {e}. File might be corrupted.")
            except IOError as e:
                print(f"Error loading records from {self.storage_file}: {e}")
        else:
            print(f"No existing records file found at {self.storage_file}. Starting with an empty system.")

# --- Example Usage ---
if __name__ == '__main__':
    # Cleanup previous records file if it exists for a fresh start in example
    if os.path.exists('student_records.json'):
        os.remove('student_records.json')

    system = StudentRecordsSystem()

    # Add some student records
    system.add_student(101, 85, 'john.doe@example.com')
    system.add_student('102', 92.5, 'jane.smith@example.com')
    system.add_student(103, 78, 'peter.jones@example.com')
```

```python
    # Try to add an invalid record
    system.add_student(106, -5, 'invalid-email')
    system.add_student(None, 70, 'test@example.com')
    system.add_student(107, 90, 'oops no spaces@example.com')

    # Display all records
    system.display_all_students()

    # Get a specific student
    student_102 = system.get_student(102)
    if student_102:
        print(f"\nRetrieved student 102: {student_102}")
    else:
        print("\nStudent 102 not found.")

    # Get top 3 students
    system.get_top_n_students(3)

    # Demonstrate persistence: Create a new system instance, and it should load previous data
    print("\n--- Creating a new system instance to demonstrate persistence ---")
    new_system = StudentRecordsSystem()
    new_system.display_all_students()

    # Add another student to the new system, which will also save it
    new_system.add_student(108, 75, 'charlie.davis@example.com')
    new_system.display_all_students()

    # Clean up the created file
    if os.path.exists('student_records.json'):
        os.remove('student_records.json')
        print(f"Cleaned up {system.storage_file}")
```

2

```python
import json
import os
from cryptography.fernet import Fernet
import base64

# --- Helper for masking ----
def mask_email(email):
    if not email or '@' not in email:
        return "********"
    parts = email.split('@')
    username = parts[0]
    domain = parts[1]
    if len(username) > 3:
        masked_username = username[:3] + '*' * (len(username) - 3)
    else:
        masked_username = '*' * len(username) # If username is very short, mask completely
    return f"{masked_username}@{domain}"

class StudentRecord:
    def __init__(self, roll_number, marks, encrypted_email_bytes):
        if not isinstance(roll_number, (int, str)) or not str(roll_number).strip():
            raise ValueError("Roll number must be a non-empty integer or string.")
        if not isinstance(marks, (int, float)) or marks < 0:
            raise ValueError("Marks must be a non-negative number.")
        if not isinstance(encrypted_email_bytes, bytes):
            raise ValueError("Encrypted email must be bytes.")

        self.roll_number = str(roll_number).strip() # Ensure roll_number is always a string and trimmed
        self.marks = marks
        self._encrypted_email = encrypted_email_bytes # Store as bytes

    def get_decrypted_email(self, fernet_cipher):
        """Decrypts and returns the email."""
```

```python
    @classmethod
    def from_dict(cls, data):
        try:
            encrypted_email_bytes = base64.urlsafe_b64decode(data['encrypted_email'].encode('utf-8'))
            return cls(data['roll_number'], data['marks'], encrypted_email_bytes)
        except Exception as e:
            print(f"Error loading StudentRecord from dict: {e}")
            # Return a record with a placeholder for the email if decryption fails
            return cls(data['roll_number'], data['marks'], b'error_loading_email')

class StudentRecordsSystem:
    def __init__(self, storage_file='student_records.json', encryption_key=None):
        self.storage_file = storage_file
        if encryption_key:
            # If a key is provided, use it (e.g., for loading existing data)
            self._key = encryption_key
        else:
            # Generate a new key if none provided (for example/first run or new system)
            self._key = Fernet.generate_key()
            print("Generated a new encryption key. In a production system, store this securely and load it consistently!")
        self._fernet = Fernet(self._key)
        self.records = {}
        self.load_records()
        print(f"Student records system initialized. Data loaded from {self.storage_file}")

    def get_encryption_key(self):
        return self._key.decode('utf-8') # Return as string for display/storage

    def add_student(self, roll_number, marks, email):
        # Validate email format before encryption
        if not isinstance(email, str) or '@' not in email or '.' not in email or ' ' in email:
            print(f"Error adding student: Email '{email}' is invalid.")
            return

        try:
            # Basic validation for roll_number and marks will be caught by StudentRecord.__init__
            encrypted_email_bytes = self._fernet.encrypt(email.strip().encode())
            student = StudentRecord(roll_number, marks, encrypted_email_bytes)
            if student.roll_number in self.records:
                print(f"Warning: Student with Roll Number {student.roll_number} already exists. Updating record.")
            self.records[student.roll_number] = student
            print(f"Added/Updated student: {student.roll_number}")
            self.save_records()
        except ValueError as e:
            print(f"Error adding student: {e}")
        except Exception as e:
            print(f"An unexpected error occurred while adding student: {e}")

    def get_student(self, roll_number):
        return self.records.get(str(roll_number).strip())

    def display_all_students(self):
        if not self.records:
            print("No student records available.")
            return
        print("\n--- All Student Records (Emails Masked) ---")
        for roll_number, student in self.records.items():
            print(f"Roll No: {student.roll_number}, Marks: {student.marks}, Email: {student.get_masked_email(self._fernet)}")
        print("-----------------------------------------")

    def get_top_n_students(self, n=3):
        if not self.records:
            print("No student records to rank.")
            return []
        sorted_students = sorted(self.records.values(), key=lambda s: s.marks, reverse=True)
        print(f"\n--- Top {n} Students (Emails Masked) ---")
        for student in sorted_students[:n]:
            print(f"Roll No: {student.roll_number}, Marks: {student.marks}, Email: {student.get_masked_email(self._fernet)}")
        print("-----------------------------------")
```

```python
    def save_records(self):
        """Saves all student records to a JSON file."""
        try:
            with open(self.storage_file, 'w') as f:
                json.dump({rn: rec.to_dict() for rn, rec in self.records.items()}, f, indent=4)
            print(f"Records saved securely to {self.storage_file}")
        except IOError as e:
            print(f"Error saving records to {self.storage_file}: {e}")

    def load_records(self):
        """Loads student records from a JSON file."""
        if os.path.exists(self.storage_file):
            try:
                with open(self.storage_file, 'r') as f:
                    data = json.load(f)
                    self.records = {rn: StudentRecord.from_dict(d) for rn, d in data.items()}
                print(f"Records loaded from {self.storage_file}")
            except json.JSONDecodeError as e:
                print(f"Error decoding JSON from {self.storage_file}: {e}. File might be corrupted.")
            except IOError as e:
                print(f"Error loading records from {self.storage_file}: {e}")
        else:
            print(f"No existing records file found at {self.storage_file}. Starting with an empty system.")

# --- Example Usage ---
if __name__ == '__main__':
    # Cleanup previous records file if it exists for a fresh start in example
    if os.path.exists('student_records.json'):
        os.remove('student_records.json')

    # Generate a key for this session. In a real app, load this from a secure source
    # and ensure it's consistent across sessions to decrypt old data.
    session_key = Fernet.generate_key()
    print(f"Using session encryption key (encoded): {session_key.decode()}")
    system = StudentRecordsSystem(encryption_key=session_key)

    # Add some student records
    system.add_student(101, 85, 'john.doe@example.com')
    system.add_student('102', 92.5, 'jane.smith@example.com')
    system.add_student(103, 78, 'peter.jones@example.com')
    system.add_student(104, 95, 'alice.brown@example.com')
    system.add_student(105, 88, 'bob.white@example.com')

    # Try to add an invalid record (validation checks included before encryption)
    system.add_student(106, -5, 'invalid-email') # Invalid marks
    system.add_student(None, 70, 'test@example.com') # Invalid roll number
    system.add_student(107, 90, 'oops no spaces@example.com') # Invalid email format

    # Display all records (emails will be masked)
    system.display_all_students()

    # Get a specific student and demonstrate decryption
    student_102 = system.get_student(102)
    if student_102:
        print(f"\nRetrieved student 102 (masked email): {student_102.get_masked_email(system._fernet)}")
        print(f"Retrieved student 102 (decrypted email): {student_102.get_decrypted_email(system._fernet)}")
    else:
        print("\nStudent 102 not found.")

    # Get top 3 students (emails will be masked)
    system.get_top_n_students(3)

    # Demonstrate persistence with encrypted data: Create a new system instance with the SAME key
    print("\n--- Creating a new system instance to demonstrate persistence with encrypted data ---")
    new_system = StudentRecordsSystem(encryption_key=session_key)
    new_system.display_all_students()

    # Add another student to the new system, which will also save it
```

```python
    # Add another student to the new system, which will also save it
    new_system.add_student(108, 75, 'charlie.davis@example.com')
    new_system.display_all_students()

    # Demonstrate what happens if you try to load with a DIFFERENT key (will fail to decrypt)
    print("\n--- Demonstrating loading with a different key (will cause decryption error) ---")
    try:
        wrong_key = Fernet.generate_key()
        print(f"Attempting to load with a different key: {wrong_key.decode()}")
        wrong_system = StudentRecordsSystem(encryption_key=wrong_key)
        wrong_system.display_all_students() # This will likely show masked 'error_loading_email' or decryption errors
    except Exception as e:
        # Fernet decryption errors are often caught internally by get_decrypted_email
        print(f"Caught an error during wrong key load/display, which is expected behavior for encrypted data: {e}")

    # Clean up the created file
    if os.path.exists('student_records.json'):
        os.remove('student_records.json')
        print(f"Cleaned up {system.storage_file}")
```

OUTPUT

```
No existing records file found at student_records.json. Starting with an empty system.
Student records system initialized. Data loaded from student_records.json
Added/Updated student: 101
Records saved securely to student_records.json
Added/Updated student: 102
Records saved securely to student_records.json
Added/Updated student: 103
Records saved securely to student_records.json
Added/Updated student: 104
Records saved securely to student_records.json
Added/Updated student: 105
Records saved securely to student_records.json
Error adding student: Marks must be a non-negative number.
Error adding student: Roll number must be a non-empty integer or string.
Error adding student: Email must be a valid string containing '@', '.', and no spaces.

--- All Student Records ---
Roll No: 101, Marks: 85, Email: john.doe@example.com
Roll No: 102, Marks: 92.5, Email: jane.smith@example.com
Roll No: 103, Marks: 78, Email: peter.jones@example.com
Roll No: 104, Marks: 95, Email: alice.brown@example.com
Roll No: 105, Marks: 88, Email: bob.white@example.com
--------------------------

Retrieved student 102: Roll No: 102, Marks: 92.5, Email: jane.smith@example.com

--- Top 3 Students ---
Roll No: 104, Marks: 95, Email: alice.brown@example.com
Roll No: 102, Marks: 92.5, Email: jane.smith@example.com
Roll No: 105, Marks: 88, Email: bob.white@example.com
---------------------

--- Creating a new system instance to demonstrate persistence ---
Records loaded from student_records.json
Student records system initialized. Data loaded from student_records.json
```


```
Retrieved student 102: Roll No: 102, Marks: 92.5, Email: jane.smith@example.com

--- Top 3 Students ---
Roll No: 104, Marks: 95, Email: alice.brown@example.com
Roll No: 102, Marks: 92.5, Email: jane.smith@example.com
Roll No: 105, Marks: 88, Email: bob.white@example.com
---------------------

--- Creating a new system instance to demonstrate persistence ---
Records loaded from student_records.json
Student records system initialized. Data loaded from student_records.json
```


```
--- All Student Records ---
Roll No: 101, Marks: 85, Email: john.doe@example.com
Roll No: 102, Marks: 92.5, Email: jane.smith@example.com
Roll No: 103, Marks: 78, Email: peter.jones@example.com
Roll No: 104, Marks: 95, Email: alice.brown@example.com
Roll No: 105, Marks: 88, Email: bob.white@example.com
--------------------------
Added/Updated student: 108
Records saved securely to student_records.json

--- All Student Records ---
Roll No: 101, Marks: 85, Email: john.doe@example.com
Roll No: 102, Marks: 92.5, Email: jane.smith@example.com
Roll No: 103, Marks: 78, Email: peter.jones@example.com
Roll No: 104, Marks: 95, Email: alice.brown@example.com
Roll No: 105, Marks: 88, Email: bob.white@example.com
Roll No: 108, Marks: 75, Email: charlie.davis@example.com
--------------------------
Cleaned up student_records.json
```

2

```
Using session encryption key (encoded): 30l52ZdU3XzyUqYNNR1w58riUE8FhenrSJyYigiJ5HU=
No existing records file found at student_records.json. Starting with an empty system.
Student records system initialized. Data loaded from student_records.json
Added/Updated student: 101
Records saved securely to student_records.json
Added/Updated student: 102
Records saved securely to student_records.json
Added/Updated student: 103
Records saved securely to student_records.json
Added/Updated student: 104
Records saved securely to student_records.json
Added/Updated student: 105
Records saved securely to student_records.json
Error adding student: Email 'invalid-email' is invalid.
Error adding student: Roll number must be a non-empty integer or string.
Error adding student: Email 'oops no spaces@example.com' is invalid.

--- All Student Records (Emails Masked) ---
Roll No: 101, Marks: 85, Email: joh*****@example.com
Roll No: 102, Marks: 92.5, Email: jan*******@example.com
Roll No: 103, Marks: 78, Email: pet********@example.com
Roll No: 104, Marks: 95, Email: ali********@example.com
Roll No: 105, Marks: 88, Email: bob******@example.com
----------------------------------------

Retrieved student 102 (masked email): jan*******@example.com
Retrieved student 102 (decrypted email): jane.smith@example.com

--- Top 3 Students (Emails Masked) ---
Roll No: 104, Marks: 95, Email: ali********@example.com
Roll No: 102, Marks: 92.5, Email: jan*******@example.com
Roll No: 105, Marks: 88, Email: bob******@example.com
-------------------------------------
```

```
--- Creating a new system instance to demonstrate persistence with encrypted data ---
Records loaded from student_records.json
Student records system initialized. Data loaded from student_records.json

--- All Student Records (Emails Masked) ---
Roll No: 101, Marks: 85, Email: joh*****@example.com
Roll No: 102, Marks: 92.5, Email: jan*******@example.com
Roll No: 103, Marks: 78, Email: pet********@example.com
Roll No: 104, Marks: 95, Email: ali********@example.com
Roll No: 105, Marks: 88, Email: bob******@example.com
----------------------------------------
Added/Updated student: 108
Records saved securely to student_records.json

--- All Student Records (Emails Masked) ---
Roll No: 101, Marks: 85, Email: joh*****@example.com
Roll No: 102, Marks: 92.5, Email: jan*******@example.com
Roll No: 103, Marks: 78, Email: pet********@example.com
Roll No: 104, Marks: 95, Email: ali********@example.com
Roll No: 105, Marks: 88, Email: bob******@example.com
Roll No: 108, Marks: 75, Email: cha**********@example.com
----------------------------------------

--- Demonstrating loading with a different key (will cause decryption error) ---
Attempting to load with a different key: b9m_UlpLcn0CexXhnJNuxp2j5RRyf8M89tuQQBSeDu8=
Records loaded from student_records.json
Student records system initialized. Data loaded from student_records.json

--- All Student Records (Emails Masked) ---
Roll No: 101, Marks: 85, Email: ********
Roll No: 102, Marks: 92.5, Email: ********
Roll No: 103, Marks: 78, Email: ********
Roll No: 104, Marks: 95, Email: ********
Roll No: 105, Marks: 88, Email: ********
Roll No: 108, Marks: 75, Email: ********
----------------------------------------
Cleaned up student_records.json
```

EXPLANATION

The prompt asks the AI to generate a program that processes student academic records such as roll number, marks, and email. The AI creates code to store, manage, and possibly display this data. We then analyze whether the code protects sensitive information properly. It is observed that personal data should be masked or encrypted to prevent unauthorized access. The prompt also highlights the importance of following privacy laws (like FERPA, GDPR principles) and ensuring ethical handling of student data.

Task 3 – AI-Generated Security Risks

PROMPT

Generate a payment processing script for an online shopping system that handles user card details and transactions. Also identify potential security risks in the code (such as plain-text storage and missing input validation) and suggest secure coding practices to prevent fraud and misuse.

CODE

```python
class PaymentProcessor:
    def __init__(self):
        # 2. In the __init__ method, initialize an empty dictionary or list to store simulated card details
        # This storage should intentionally not be encrypted (plain-text storage)
        self.card_details_storage = []
        print("PaymentProcessor initialized. WARNING: Card details are stored in plain text.")

    def process_payment(self, card_number, expiry_date, cvv, amount):
        # 3a. Prints the full card_number, expiry_date, and cvv to the console as part of simulating a log or transaction record.
        print("\n--- Processing Payment ---")
        print(f"Logging sensitive data: Card Number: {card_number}, Expiry Date: {expiry_date}, CVV: {cvv}")
        # No input validation for format, length, or validity

        # 3b. Simulates a payment transaction by simply printing a message indicating a successful payment for the given amount.
        print(f"Simulating payment: Successfully processed ${amount:.2f} transaction.")

        # 3c. Calls another method to store the card details.
        self.store_card_details(card_number, expiry_date, cvv)
        print("Payment processed and card details stored (insecurely).")

    def store_card_details(self, card_number, expiry_date, cvv):
        # 4a. Stores the card_number, expiry_date, and cvv directly into self.card_details_storage without any encryption, tokenization, or hashing.
        card_info = {
            'card_number': card_number,
            'expiry_date': expiry_date,
            'cvv': cvv
        }
        self.card_details_storage.append(card_info)
        # 4b. Do not include any validation for the format, length, or validity of the parameters.
        print(f"Stored card details: {card_info} in plain text.")


# --- Example Usage ---
if __name__ == '__main__':
```

```python
# --- Example Usage ---
if __name__ == '__main__':
    print("\n--- Demonstrating Vulnerable Payment Processor ---")
    processor = PaymentProcessor()

    # Example 1: Valid-looking card details
    print("\nAttempting payment with valid-looking card details:")
    processor.process_payment('1234-5678-9012-3456', '12/25', '123', 100.50)

    # Example 2: Invalid-looking card details (no validation will occur)
    print("\nAttempting payment with invalid-looking card details:")
    processor.process_payment('SHORT', 'BAD', 'XX', 25.00)

    # Example 3: Another valid-looking card
    print("\nAttempting payment with another valid-looking card details:")
    processor.process_payment('9876-5432-1098-7654', '01/24', '456', 75.25)

    print("\n--- Stored Card Details (Plain Text) ---")
    for i, card in enumerate(processor.card_details_storage):
        print(f"Card {i+1}: {card}")
    print("WARNING: All card details above are stored in plain text due to intentional vulnerabilities.")
```

OUTPUT

```
--- Demonstrating Vulnerable Payment Processor ---
PaymentProcessor initialized. WARNING: Card details are stored in plain text.

Attempting payment with valid-looking card details:

--- Processing Payment ---
Logging sensitive data: Card Number: 1234-5678-9012-3456, Expiry Date: 12/25, CVV: 123
Simulating payment: Successfully processed $100.50 transaction.
Stored card details: {'card_number': '1234-5678-9012-3456', 'expiry_date': '12/25', 'cvv': '123'} in plain text.
Payment processed and card details stored (insecurely).

Attempting payment with invalid-looking card details:

--- Processing Payment ---
Logging sensitive data: Card Number: SHORT, Expiry Date: BAD, CVV: XX
Simulating payment: Successfully processed $25.00 transaction.
Stored card details: {'card_number': 'SHORT', 'expiry_date': 'BAD', 'cvv': 'XX'} in plain text.
Payment processed and card details stored (insecurely).

Attempting payment with another valid-looking card details:

--- Processing Payment ---
Logging sensitive data: Card Number: 9876-5432-1098-7654, Expiry Date: 01/24, CVV: 456
Simulating payment: Successfully processed $75.25 transaction.
Stored card details: {'card_number': '9876-5432-1098-7654', 'expiry_date': '01/24', 'cvv': '456'} in plain text.
Payment processed and card details stored (insecurely).

--- Stored Card Details (Plain Text) ---
Card 1: {'card_number': '1234-5678-9012-3456', 'expiry_date': '12/25', 'cvv': '123'}
Card 2: {'card_number': 'SHORT', 'expiry_date': 'BAD', 'cvv': 'XX'}
Card 3: {'card_number': '9876-5432-1098-7654', 'expiry_date': '01/24', 'cvv': '456'}
WARNING: All card details above are stored in plain text due to intentional vulnerabilities.
```

- **Data Integrity:** Ensuring data is accurate, consistent, and reliable.
- **Security by Design:** Integrating security considerations from the initial stages of development.
- **Auditability:** Allowing for clear tracking and verification of all actions and transactions.
- **Least Privilege:** Granting users and systems only the minimum necessary access rights.
- **Error Handling:** Implementing robust mechanisms to manage and log errors securely.
- **Regular Security Audits:** Conducting periodic assessments to identify and mitigate vulnerabilities.
- **Ethical Responsibility:** Fostering a culture of accountability and ethical conduct among developers.
- **Compliance:** Adhering to relevant legal and regulatory requirements.
- **Privacy:** Protecting sensitive user and financial information.
- **Immutable Transaction Records:** Ensuring that once recorded, financial transactions cannot be altered.

EXPLANATION

Given prompt to AI to generate a payment processing script for an online shopping system that handles card details and transactions.

The AI creates code to process payments using user financial information.We then analyze the generated code to identify security risks such as storing card data in plain text and lack of input validation.

These issues can lead to data theft, fraud, and system attacks.

Task 4 – Accountability in Human–AI Collaboration

PROMPT

Generate code for an autonomous traffic signal control system that manages traffic flow using AI. Also analyze who should be accountable if a software error causes accidents, and discuss the responsibilities of AI developers, software engineers, and government authorities.

CODE

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# --- AI Project Example: Simple Customer Churn Prediction ---
# This example demonstrates a basic machine learning workflow for predicting customer churn.
# In a real-world scenario, data would be much larger and more complex,
# and model tuning, feature engineering, and robust deployment would be critical.

# 1. Generate Synthetic Data
# In a real startup/academic project, this would be loaded from a database or file
print("Generating synthetic customer data...")
data = {
    'age': [random.randint(20, 60) for _ in range(1000)],
    'monthly_bill': [random.uniform(30, 150) for _ in range(1000)],
    'data_usage_gb': [random.uniform(5, 100) for _ in range(1000)],
    'contract_type': [random.choice(['month-to-month', 'one-year', 'two-year']) for _ in range(1000)],
    'tech_support': [random.choice([0, 1]) for _ in range(1000)], # 0=No, 1=Yes
    'churn': [random.choice([0, 1]) for _ in range(1000)] # Target variable: 0=No Churn, 1=Churn
}
df = pd.DataFrame(data)

# Introduce some correlation to make churn more 'predictable' for demonstration
df['churn'] = df.apply(lambda row: 1 if (row['monthly_bill'] > 100 and row['data_usage_gb'] < 20 and row['contract_type'] == 'month-to-month' and ro
df['churn'] = df.apply(lambda row: 0 if (row['contract_type'] == 'two-year' and random.random() < 0.8) else row['churn'], axis=1)


# 2. Preprocessing
# Convert categorical features into numerical using one-hot encoding
print("Preprocessing data (one-hot encoding categorical features)...")
df = pd.get_dummies(df, columns=['contract_type'], drop_first=True)

# Define features (X) and target (y)

print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

# 3. Model Training
print("Training a RandomForestClassifier model...")
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("Model training complete.")

# 4. Model Evaluation
print("Evaluating model performance...")
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"\nModel Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(report)

# 5. Example Prediction
print("\nMaking an example prediction...")
# Create a new 'customer' with features similar to the original data
# Be careful to match the column order and encoding of the training data
new_customer_data = pd.DataFrame([{
    'age': 35,
    'monthly_bill': 110.0,
    'data_usage_gb': 15.0,
    'tech_support': 0,
    'contract_type_one-year': 0, # Assuming 'month-to-month' for simplicity after one-hot encoding
    'contract_type_two-year': 0
```

```python
# Ensure the new customer data has the same columns as the training data, in the same order
# (Important for one-hot encoded features)
new_customer_data = new_customer_data.reindex(columns=X_train.columns, fill_value=0)

prediction = model.predict(new_customer_data)
prediction_proba = model.predict_proba(new_customer_data)

print(f"New customer data: {new_customer_data.iloc[0].to_dict()}")
print(f"Predicted Churn (0=No, 1=Yes): {prediction[0]}")
print(f"Prediction Probability (No Churn, Churn): {prediction_proba[0]}")

if prediction[0] == 1:
    print("This customer is predicted to churn. Proactive measures might be needed!")
else:
    print("This customer is predicted to not churn.")
```

OUTPUT

```
TrafficLight class defined.
Intersection class defined.
AITrafficController class defined.
Intersection initialized with lights: ['North', 'South', 'East', 'West']
AITrafficController initialized.

--- Starting Traffic Simulation ---

Simulation Time: 0.0s
Traffic Density: {'North': 23, 'South': 86, 'East': 84, 'West': 76}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 1.0s)
Light South: GREEN (Time in state: 1.0s)
Light East: RED (Time in state: 1.0s)
Light West: RED (Time in state: 1.0s)
-----------------------------------

Simulation Time: 1.0s
Traffic Density: {'North': 42, 'South': 94, 'East': 80, 'West': 78}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 2.0s)
Light South: GREEN (Time in state: 2.0s)
Light East: RED (Time in state: 2.0s)
Light West: RED (Time in state: 2.0s)
-----------------------------------

Simulation Time: 2.0s
Traffic Density: {'North': 84, 'South': 78, 'East': 19, 'West': 100}
AI Decision based on density: West to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 3.0s)
Light South: RED (Time in state: 1.0s)
```

```
-----------------------------------

Simulation Time: 3.0s
Traffic Density: {'North': 30, 'South': 93, 'East': 1, 'West': 69}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 4.0s)
Light South: GREEN (Time in state: 1.0s)
Light East: RED (Time in state: 4.0s)
Light West: RED (Time in state: 1.0s)
-----------------------------------

Simulation Time: 4.0s
Traffic Density: {'North': 86, 'South': 95, 'East': 92, 'West': 61}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 5.0s)
Light South: GREEN (Time in state: 2.0s)
Light East: RED (Time in state: 5.0s)
Light West: RED (Time in state: 2.0s)
-----------------------------------

Simulation Time: 5.0s
Traffic Density: {'North': 24, 'South': 57, 'East': 10, 'West': 39}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 6.0s)
Light South: GREEN (Time in state: 3.0s)
Light East: RED (Time in state: 6.0s)
Light West: RED (Time in state: 3.0s)
-----------------------------------

Simulation Time: 6.0s
Traffic Density: {'North': 38, 'South': 22, 'East': 25, 'West': 66}
```

```
--- Current Traffic Light States ---
Light North: RED (Time in state: 7.0s)
Light South: RED (Time in state: 1.0s)
Light East: RED (Time in state: 7.0s)
Light West: GREEN (Time in state: 1.0s)
------------------------------------

Simulation Time: 7.0s
Traffic Density: {'North': 1, 'South': 100, 'East': 10, 'West': 67}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 8.0s)
Light South: GREEN (Time in state: 1.0s)
Light East: RED (Time in state: 8.0s)
Light West: RED (Time in state: 1.0s)
------------------------------------

Simulation Time: 8.0s
Traffic Density: {'North': 11, 'South': 14, 'East': 75, 'West': 4}
AI Decision based on density: East to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 9.0s)
Light South: RED (Time in state: 1.0s)
Light East: GREEN (Time in state: 1.0s)
Light West: RED (Time in state: 2.0s)
------------------------------------

Simulation Time: 9.0s
Traffic Density: {'North': 24, 'South': 74, 'East': 16, 'West': 30}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 10.0s)
Light South: GREEN (Time in state: 1.0s)
Light East: RED (Time in state: 1.0s)
```

```
-----------------------------------

Simulation Time: 11.0s
Traffic Density: {'North': 12, 'South': 78, 'East': 63, 'West': 58}
AI Decision based on density: South to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 12.0s)
Light South: GREEN (Time in state: 1.0s)
Light East: RED (Time in state: 3.0s)
Light West: RED (Time in state: 1.0s)
-----------------------------------

Simulation Time: 12.0s
Traffic Density: {'North': 35, 'South': 81, 'East': 88, 'West': 76}
AI Decision based on density: East to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 13.0s)
Light South: RED (Time in state: 1.0s)
Light East: GREEN (Time in state: 1.0s)
Light West: RED (Time in state: 2.0s)
-----------------------------------

Simulation Time: 13.0s
Traffic Density: {'North': 84, 'South': 47, 'East': 39, 'West': 4}
AI Decision based on density: North to GREEN

--- Current Traffic Light States ---
Light North: GREEN (Time in state: 1.0s)
Light South: RED (Time in state: 2.0s)
Light East: RED (Time in state: 1.0s)
Light West: RED (Time in state: 3.0s)
-----------------------------------

Simulation Time: 14.0s
Traffic Density: {'North': 87, 'South': 10, 'East': 98, 'West': 87}
```

```
Light West: RED (Time in state: 16.0s)
-----------------------------------
...
Simulation Time: 27.0s
Traffic Density: {'North': 29, 'South': 3, 'East': 15, 'West': 63}
AI Decision based on density: West to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 1.0s)
Light South: RED (Time in state: 2.0s)
Light East: RED (Time in state: 3.0s)
Light West: GREEN (Time in state: 1.0s)
-----------------------------------

Simulation Time: 28.0s
Traffic Density: {'North': 39, 'South': 6, 'East': 37, 'West': 100}
AI Decision based on density: West to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 2.0s)
Light South: RED (Time in state: 3.0s)
Light East: RED (Time in state: 4.0s)
Light West: GREEN (Time in state: 2.0s)
-----------------------------------

Simulation Time: 29.0s
Traffic Density: {'North': 26, 'South': 94, 'East': 86, 'West': 99}
AI Decision based on density: West to GREEN

--- Current Traffic Light States ---
Light North: RED (Time in state: 3.0s)
Light South: RED (Time in state: 4.0s)
Light East: RED (Time in state: 5.0s)
Light West: GREEN (Time in state: 3.0s)
-----------------------------------

--- Simulation Ended ---
```

Yes, responsibility *should* lie with AI tool developers for several key reasons, as outlined in the previous response:

1. **Primary Responsibility for Code Quality and Safety:** AI tool developers are at the forefront of creating the algorithms and software that power these systems. They are directly responsible for the quality, reliability, and safety of the code they produce. This includes rigorous testing, thorough validation, and ensuring adherence to strict coding standards.
2. **Handling Edge Cases and Biases:** Developers must ensure the AI logic is sound, can effectively handle unforeseen edge cases, and does not inadvertently introduce biases or unpredictable behaviors that could lead to dangerous situations. Their design choices directly impact the system's ethical and safety performance.
3. **Addressing Known Issues:** If an accident is traceable to poor design, inadequate testing, known bugs that were not addressed, or a failure to follow established safety protocols during development, the individual developers or their organization could be held accountable.
4. **Implementing Ethical AI Principles:** AI developers have a crucial role in embedding ethical AI principles (such as fairness, transparency, and accountability) into the system's design and implementation from the very beginning.

While accountability is often shared among multiple parties (manufacturers, government agencies, operators), the AI tool developers form the foundational layer of responsibility due to their direct influence over the system's core functionality and safety.

EXPLANATION

The prompt asks the AI to generate code for an autonomous traffic signal control system that manages traffic automatically.

The AI produces logic to control signals based on traffic conditions.We then analyze what happens if a software error causes accidents.

The discussion focuses on accountability among AI developers, software engineers, and government authorities.It is concluded that responsibility is shared, as developers build the AI, engineers implement and test it, and authorities approve and deploy the system.

Task 5 – Limitations of AI in Critical Systems

PROMPT

Generate code for an aircraft maintenance fault detection system that identifies potential issues based on input data. Also explain the limitations of using AI-generated code in safety critical systems, and describe the need for human validation, testing, and certification.

CODE

```python
class AircraftFaultDetector:
    def __init__(self, normal_operating_ranges, anomaly_thresholds):
        self.normal_ranges = normal_operating_ranges
        self.anomaly_thresholds = anomaly_thresholds
        print("AircraftFaultDetector initialized with predefined ranges and thresholds.")

    def detect_faults(self, sensor_data):
        """
        Detects potential faults based on current sensor data and predefined thresholds.
        sensor_data example: {'engine_temp': 120, 'fuel_pressure': 55, 'vibration_level': 1.5}
        """
        detected_faults = []
        for parameter, value in sensor_data.items():
            if parameter in self.normal_ranges:
                min_val, max_val = self.normal_ranges[parameter]
                if not (min_val <= value <= max_val):
                    detected_faults.append(f"Anomaly: {parameter} ({value}) is outside normal range [{min_val}-{max_val}].")

            if parameter in self.anomaly_thresholds:
                threshold = self.anomaly_thresholds[parameter]
                if value > threshold: # Assuming a 'higher than threshold' indicates an anomaly
                    detected_faults.append(f"Critical: {parameter} ({value}) exceeds anomaly threshold ({threshold}).")

        return detected_faults

# Example Usage:
if __name__ == '__main__':
    # Define normal operating ranges for various parameters
    normal_ranges = {
        'engine_temp': (100, 150),  # Celsius
        'fuel_pressure': (40, 60),  # PSI
        'vibration_level': (0.1, 2.0), # G-force
        'hydraulic_pressure': (2000, 3000) # PSI
    }
```

```python
# Define anomaly thresholds (e.g., values above which are critical)
anomaly_thresholds = {
    'engine_temp': 160,
    'vibration_level': 3.0
}

detector = AircraftFaultDetector(normal_ranges, anomaly_thresholds)

# Simulate various sensor data inputs
data_scenario_1 = {
    'engine_temp': 130,
    'fuel_pressure': 50,
    'vibration_level': 1.0,
    'hydraulic_pressure': 2500
}
print("\n--- Scenario 1: Normal Operation ---")
faults = detector.detect_faults(data_scenario_1)
if faults:
    print("Detected Faults:", faults)
else:
    print("No faults detected.")

data_scenario_2 = {
    'engine_temp': 155, # Slightly above normal range, but below anomaly threshold
    'fuel_pressure': 35, # Below normal range
    'vibration_level': 2.5,
    'hydraulic_pressure': 2800
}
print("\n--- Scenario 2: Minor Anomalies ---")
faults = detector.detect_faults(data_scenario_2)
if faults:
    print("Detected Faults:", faults)
else:
    print("No faults detected.")
```


```python
    print("Detected Faults:", faults)
else:
    print("No faults detected.")
```


```python
data_scenario_3 = {
    'engine_temp': 170, # Above anomaly threshold
    'fuel_pressure': 55,
    'vibration_level': 3.5, # Above anomaly threshold
    'hydraulic_pressure': 1900 # Below normal range
}
print("\n--- Scenario 3: Critical Faults ---")
faults = detector.detect_faults(data_scenario_3)
if faults:
    print("Detected Faults:", faults)
else:
    print("No faults detected.")
```

OUTPUT

```
AircraftFaultDetector initialized with predefined ranges and thresholds.

--- Scenario 1: Normal Operation ---
No faults detected.

--- Scenario 2: Minor Anomalies ---
Detected Faults: ['Anomaly: engine_temp (155) is outside normal range [100-150].', 'Anomaly: fuel_pressure (35) is outside normal range [40-60].', 'Anomaly: vibration_level (2.5) i

--- Scenario 3: Critical Faults ---
Detected Faults: ['Anomaly: engine_temp (170) is outside normal range [100-150].', 'Critical: engine_temp (170) exceeds anomaly threshold (160).', 'Anomaly: vibration_level (3.5) i
```

No, AI-generated code generally *cannot* be fully trusted without expert validation, especially in safety-critical systems. As I explained earlier, several factors necessitate human oversight:

1. **Unpredictable Behavior and Black Box Nature:** AI models can produce code that is difficult to understand or verify, making it challenging to predict behavior, especially in edge cases.
2. **Lack of Formal Verification:** Unlike traditional software engineering for safety-critical systems, AI-generated code doesn't come with formal proofs of correctness.
3. **Potential for Errors and Vulnerabilities:** AI can introduce subtle bugs, security flaws, or inefficiencies that human experts are better equipped to identify and correct.
4. **Absence of Ethical and Legal Accountability:** The ultimate responsibility for system safety lies with human engineers, developers, and certifiers.
5. **Requirement for Rigorous Testing and Certification:** All code for safety-critical systems must undergo extensive human-designed testing, validation against requirements, and often formal certification by regulatory bodies—processes that cannot be fully automated by AI alone.

EXPLANATION

We asked the AI to generate code for an aircraft fault detection system.
The AI created logic to identify possible issues in aircraft maintenance.
We understood that AI-generated code cannot be fully trusted in critical systems without expert checking.
Finally, we concluded that human validation, testing, and certification are necessary for safety.

Task 6 – Ethical Use of AI-Generated Code

PROMPT

Generate code for an academic or startup project using AI, and explain the ethical issues of using AI-generated code, including risks of not understanding it, copyright or license violations, and best practices like proper citation and manual verification.

CODE

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# --- AI Project Example: Simple Customer Churn Prediction ---
# This example demonstrates a basic machine learning workflow for predicting customer churn.
# In a real-world scenario, data would be much larger and more complex,
# and model tuning, feature engineering, and robust deployment would be critical.

# 1. Generate Synthetic Data
# In a real startup/academic project, this would be loaded from a database or file
print("Generating synthetic customer data...")
data = {
    'age': [random.randint(20, 60) for _ in range(1000)],
    'monthly_bill': [random.uniform(30, 150) for _ in range(1000)],
    'data_usage_gb': [random.uniform(5, 100) for _ in range(1000)],
    'contract_type': [random.choice(['month-to-month', 'one-year', 'two-year']) for _ in range(1000)],
    'tech_support': [random.choice([0, 1]) for _ in range(1000)], # 0=No, 1=Yes
    'churn': [random.choice([0, 1]) for _ in range(1000)] # Target variable: 0=No Churn, 1=Churn
}
df = pd.DataFrame(data)

# Introduce some correlation to make churn more 'predictable' for demonstration
df['churn'] = df.apply(lambda row: 1 if (row['monthly_bill'] > 100 and row['data_usage_gb'] < 20 and row['contract_type'] == 'month-to-month' and row['tech_support'] == 0 an
df['churn'] = df.apply(lambda row: 0 if (row['contract_type'] == 'two-year' and random.random() < 0.8) else row['churn'], axis=1)


# 2. Preprocessing
# Convert categorical features into numerical using one-hot encoding
print("Preprocessing data (one-hot encoding categorical features)...")
df = pd.get_dummies(df, columns=['contract_type'], drop_first=True)
```

```python
# Define features (X) and target (y)
X = df.drop('churn', axis=1)
y = df['churn']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training data shape: {X_train.shape}")
print(f"Testing data shape: {X_test.shape}")

# 3. Model Training
print("Training a RandomForestClassifier model...")
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("Model training complete.")

# 4. Model Evaluation
print("Evaluating model performance...")
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"\nModel Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(report)

# 5. Example Prediction
print("\nMaking an example prediction...")
# Create a new 'customer' with features similar to the original data
# Be careful to match the column order and encoding of the training data
new_customer_data = pd.DataFrame([{
    'age': 35,
    'monthly_bill': 110.0,
```

```python
# Ensure the new customer data has the same columns as the training data, in the same order
# (Important for one-hot encoded features)
new_customer_data = new_customer_data.reindex(columns=X_train.columns, fill_value=0)

prediction = model.predict(new_customer_data)
prediction_proba = model.predict_proba(new_customer_data)

print(f"New customer data: {new_customer_data.iloc[0].to_dict()}")
print(f"Predicted Churn (0=No, 1=Yes): {prediction[0]}")
print(f"Prediction Probability (No Churn, Churn): {prediction_proba[0]}")

if prediction[0] == 1:
    print("This customer is predicted to churn. Proactive measures might be needed!")
else:
    print("This customer is predicted to not churn.")
```

OUTPUT

```
Generating synthetic customer data...
Preprocessing data (one-hot encoding categorical features)...
Training data shape: (800, 6)
Testing data shape: (200, 6)
Training a RandomForestClassifier model...
Model training complete.
Evaluating model performance...

Model Accuracy: 0.6900

Classification Report:
              precision    recall  f1-score   support

           0       0.74      0.76      0.75       121
           1       0.61      0.58      0.60        79

    accuracy                           0.69       200
   macro avg       0.67      0.67      0.67       200
weighted avg       0.69      0.69      0.69       200


Making an example prediction...
New customer data: {'age': 35.0, 'monthly_bill': 110.0, 'data_usage_gb': 15.0, 'tech_support': 0.0, 'contract_type_one-year': 0.0, 'contract_type_two-year': 0.0
Predicted Churn (0=No, 1=Yes): 1
Prediction Probability (No Churn, Churn): [0.21 0.79]
This customer is predicted to churn. Proactive measures might be needed!
```

Ethical Considerations for Using AI-Generated Code

Using AI-generated code offers efficiency but introduces several critical ethical and practical challenges:

1. **Risks of Not Understanding AI-Generated Code:**
   - **Security Vulnerabilities:** AI might generate code with subtle security flaws that are difficult for a human to spot without a deep understanding of the code's logic and underlying libraries. This could lead to exploitable weaknesses in applications, especially in sensitive domains like finance or personal data.
   - **Bugs and Incorrect Logic:** The AI might produce code that contains logical errors or unexpected behavior due to misunderstandings of complex requirements or edge cases. Without manual verification and thorough testing by a human who understands the problem domain, these bugs can propagate and cause significant issues.
   - **Maintenance Challenges:** If developers don't comprehend the generated code, maintaining, debugging, or extending it becomes extremely difficult, increasing technical debt and project costs.

extremely difficult, increasing technical debt and project costs.

2. **Copyright or License Violations:**
    - **Data Provenance:** AI models are trained on vast datasets, often scraped from the internet, which may include copyrighted or permissively licensed code (e.g., GPL, MIT). When the AI generates code, it might inadvertently reproduce patterns, snippets, or even entire functions that infringe on existing copyrights or violate the terms of specific open-source licenses.
    - **Legal Liability:** If an organization or individual uses AI-generated code that contains copyrighted material or violates a license, they could face legal action, including injunctions, fines, and reputational damage. Determining the original source and license of AI-generated code snippets can be nearly impossible.
    - **Attribution Issues:** Many licenses require attribution to the original authors. AI-generated code typically lacks such attribution, making compliance challenging or impossible.

3. **Best Practices for Using AI-Generated Code:**
    - **Manual Verification and Review:** Treat AI-generated code as a suggestion or a first draft. It must undergo rigorous human review by experienced developers who understand the problem domain, security implications, and code quality standards.
    - **Thorough Testing:** Implement comprehensive unit, integration, and end-to-end tests for all AI-generated code. This helps catch logical errors and unexpected behaviors.
    - **Understanding Core Logic:** Developers should strive to understand the 'why' behind the AI's suggestions, not just copy-paste. This involves familiarizing themselves with the algorithms, data structures, and design patterns used.
    - **Citation and Provenance Tracking:** If an AI tool provides a mechanism for citing sources, use it. If not, treat AI-generated code similarly to code found on platforms like Stack Overflow – understand it, adapt it, and consider adding comments about its origin (e.g., "Generated by [AI tool] and human-reviewed").
    - **Security Audits:** Integrate AI-generated code into existing security audit processes. Use static analysis tools, dynamic analysis tools, and manual security reviews.
    - **License Compliance Checks:** For commercial or open-source

EXPLANATION

I asked the AI to generate code for an academic or startup project.
The AI provided ready-made code solutions.
We understood that using AI code without understanding can cause errors and ethical issues.
Finally, we concluded that proper citation, verification, and responsible use are important.